

Dynamic opposite learning enhanced dragonfly algorithm for solving large-scale flexible job shop scheduling problem

Dongsheng Yang^a, Mingliang Wu^a, Di Li^b, Yunlang Xu^c, Xianyu Zhou^a, Zhile Yang^{d,*}

^a Intelligent Electrical Science and Technology Research Institute, Northeastern University, Shenyang, China

^b School of Mechanical and Automotive Engineering, South China University of Technology, Guangzhou, China

^c State Key Laboratory of ASIC and System, School of Microelectronics, Fudan University, Shanghai, China

^d Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China

ARTICLE INFO

Article history:

Received 11 July 2021

Received in revised form 6 October 2021

Accepted 24 November 2021

Available online 8 December 2021

Keywords:

Large-scale flexible job shop scheduling problem

Dragonfly algorithm

Dynamic opposite learning

Global optimization

Makespan

ABSTRACT

Flexible job shop scheduling problem (FJSP) has attracted many research interests, in particular for meta-heuristic algorithm (MA) developers due to the superior optimization performance. Dragonfly algorithm (DA) is one of recent and popular MA approaches. However, it is inevitable for DA to be trapped into local optima, especially when dealing with the complex large-scale flexible job shop scheduling problem (LSFJSP). In this paper, an improved DA, adopting a dynamic opposite learning (DOL) strategy, is proposed (namely DOLDA) to solve the LSFJSP. DOL strategy is embedded into the population initialization stage and the generation jumping stage to raise the search ability of DA. This paper uses a T-test to testify whether there are differences between the proposed algorithm and other comparison algorithms, comparison results indicate that DOLDA shows noticeable differences with other algorithms that explain the effectiveness and innovation of the proposed algorithm. The jump rate is an important parameter that determines the probability of algorithms escaping from the algorithms' local optimal solution. This paper also considers the jump rate analysis experiments to maximize the power of the DOLDA. 28 test functions from CEC 2013 and CEC 2014 are applied to verify the performance of DOLDA, test results reveal that DOLDA owns strong search ability in coping with almost all test functions. The DOLDA also is applied to solve 15 LSFJSP instances generated by Brardimate rule, the results obtained show that DOLDA can efficiently achieve a better solution on the LSFJSP compare to compared algorithms.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

As one of the key issues in manufacturing, production, and energy industries, scheduling problems have received more and more attention in recent years [1–3]. Among them, the job shop scheduling problem (JSP) as an important schedule problem is significantly proposed in “Industrial Scheduling” of Muth and Thompson in literature [4], and literature [5] proposed three fundamental problems about JSP. Since then, JSP has become a main research object in the scheduling field [6]. In recent 20 years, although many authors have tested and verified their methods in related reports or literature, none of researchers can put forward a representative algorithm to prove the JSP optimal [7] due to its mixed-integer combinational optimization characteristics. With the constant developments and rising personalized demand of the manufacturing industry, the flexibility gradually is taken into the JSP so that JSP is extended and form a new scheduling

problem: flexible job shop scheduling problem (FJSP) [8,9]. FJSP contains two sub-problems: machine assignment and operation sequencing [10,11]. It is necessary to select the machine used in each operation while determining the processing sequence of each operation of each job [12,13]. It makes the solving procedure more complicated due to the addition of machine selection problem [14–16].

Generally speaking, global optimization algorithms consist of two classes [17,18]: deterministic mathematical programming methods and meta-heuristic algorithms (MAs). Deterministic mathematical programming methods are not appropriate for solving FJSP due to the high computational cost for high-complexity of FJSP [19–21]. As one of the approximate algorithms, MAs can escape from local optima by temporarily accepting solutions of lower quality so that MAs are capable in obtaining comparatively better results in terms of solution time and accuracy [22,23]. Thus it is more proper to employ MAs to produce a reasonable schedule in the acceptable execution time instead of seeking an exact solution at many costs [24,25].

* Corresponding author.

E-mail address: zl.yang@siat.ac.cn (Z. Yang).

Up to now, a batch of MAs have been utilized to solve FJSP that including genetic algorithm (GA), artificial bee colony algorithm (ABC), differential evolution (DE), tabu search (TS), teaching learning-based optimization (TLBO), dragonfly algorithm (DA), and so on [26–30]. Among them, DA is a powerful and efficient MAs proposed in [31] which has been successfully applied in many fields. Suresh et al. [32] utilized DA to solve static economic dispatch incorporating solar energy. Philip et al. [33] applied DA into the wireless node localization as DA can provide precise information in wireless node localization experiments. Jafari et al. [34] employed DA to calculate the stress distribution based on Lekhnitskii's analytical solution.

Although DA has shown excellent optimization results for the above problems, it is difficult to achieve perfection on some more challenging problems [35]. Thus, it is critical to improve the performance of DA. There are various existing strategies for improving DA. Zhong et al. [36] found the wind-driven optimization algorithm (WDO) possesses the advantage of fast convergence speed and excellent local search capability and proposed a new technology named WDDA that utilizes the advantage of WDO to improved weak search ability and slow iteration period of dragonfly algorithm. Dai et al. [37] presented an improved dragonfly algorithm with higher exploitation ability (IEDA) that used a division of labor strategy to divide dragonfly individuals into exploration and exploitation populations. Debnath et al. [38] designed a hybrid memory-based dragonfly algorithm with differential evolution (DADE), which introduced a memory element record the best solution in a particular iteration when proceeded with DE to enhance population diversity and increase the probability of reaching the optimal global solution. Li et al. [39] combined the adaptive learning factor and differential evolution strategy with the dragonfly algorithm to choose the optimal parameters of support vector machine when making wind power forecasts. Cui et al. [40] proposed an excellent optimization algorithm for feature selection called hybrid improved dragonfly algorithm (HIDA), which integrated the mRMR into the dragonfly algorithm to obtain more accurate classification results. Although the above improvement strategies have improved the solving ability of DA to some extent for their problem, it is challenging when faced with FJSP, especially LSFJSP. The solution space of the LSFJSP is enormous, and the computational cost is also very expensive. It is tight for the previous research to balance exploitation and exploration when faced with the LSFJSP.

In order to solve LSFJSP accurately and efficiently, this paper is inspired by the dynamic opposite learning (DOL) strategy [41] and develops a brand new variant of DA named dynamic opposite learning dragonfly algorithm (DOLDA), which can effectively balance the exploitation and exploration process of the algorithm to address the LSFJSP. The highlights of this work mainly include:

- (1) The existing FJSP contains a relatively small number of machines and jobs, which cannot meet the increasingly complex industrial needs. For this reason, this paper designs 15 LSFJSP instances and solves them.
- (2) Considering the high-complexity of the LSFJSP, this paper designs a novel encoding and decoding principle that can effectively reduce memory usage of the algorithm and increase computing speed.
- (3) To solve the high-dimensional LSFJSP, we propose a variant of DA called DOLDA that integrate DOL strategy into the DA to enhance DA's searching ability.
- (4) 28 test functions that from CEC 2013 and CEC 2014 are selected to test the DOLDA. These test functions include unimodal function, multimodal function, and hybrid function that can test exploration ability and exploitation ability of algorithm in particular.

The remainder of this paper is organized as follows. The details of FJSP and LSFJSP are elaborated in Section 2 firstly. Next, we introduce DA and DOL strategy as the preparations for DOLDA's proposal in Section 3, followed by the proposal of DOLDA in Section 4. Then, extensive experiments are conducted to investigate the performance of DOLDA in Section 5. Finally, conclude the paper in Section 6.

2. The FJSP and LSFJSP

FJSP is an important scheduling problem that its pursuit is to reasonably deploy the processing order of jobs on the machine under certain constraints to optimize one or more scheduling indicators [6]. A $n \times m$ FJSP can be expounded as follow. A set of n jobs $J = \{J_1, J_2, \dots, J_n\}$ are processed on a set of m machines $M = \{M_1, M_2, \dots, M_m\}$. Each job J_i is formed by a sequence of n_i operations O_{ij} ($j = 1, 2, \dots, n_i$) to be performed one after the other in the advanced sequence. The solving of the FJSP is to determine the most appropriate machine for each operation (called machine selection) and the sequence of the operations on machines (called operation sequencing) [42].

2.1. The categories of FJSP

FJSP breaks the constraints of machine selection and allows the operation to be processed on one or more machines. According to the degree of flexibility, the FJSP can be classified into partial FJSP (P-FJSP) and total FJSP (T-FJSP). Relationships among JSP, P-FJSP and T-FJSP are shown in Fig. 1. In Fig. 1, there is only one available machine for the operation in the JSP, so that machine selection does not exist on JSP. On the other hand, machine selection in the T-FJSP is more flexible than in the P-FJSP due to an operation can be processed on total machines rather than partial machines.

2.2. The mathematical model of LSFJSP

LSFJSP is the result of FJSP's expansion in quantity, so it has the same mathematical model as FJSP. The LSFJSP mathematical model consists of constraints and objective functions. Without loss of generality, the mathematical constraints of LSFJSP model can be specified with the following [43]:

(1) Once processing is in progress, it cannot be stopped, and the completion time of the j th operation of the i th job O_{ij} is the sum of its start time and processing time.

$$F_{ijk} = B_{ijk} + P_{ijk} \quad (1)$$

where F_{ijk} , B_{ijk} , P_{ijk} implies the completion time, the starting time, and the processing time of the j th operation of the i th job O_{ij} on machine k , respectively.

(2) The start time of the j th operation of the i th job O_{ij} on the machine k is determined by the completion time of the previous operation q on the machine and the completion time of the previous operation of the job.

$$B_{ijk} = \max\{B_{(i-1)qk} + P_{(i-1)qk}, F_{i(j-1)(k-1)}\} \quad (2)$$

(3) Machine k shuts down after finishing the last operation on it.

$$FT_k = F_{lk} \quad (3)$$

where FT_k denotes the downtime of the machine k , l represents the last job on machine k , F_{lk} indicates the completion time of the last job on machine k .

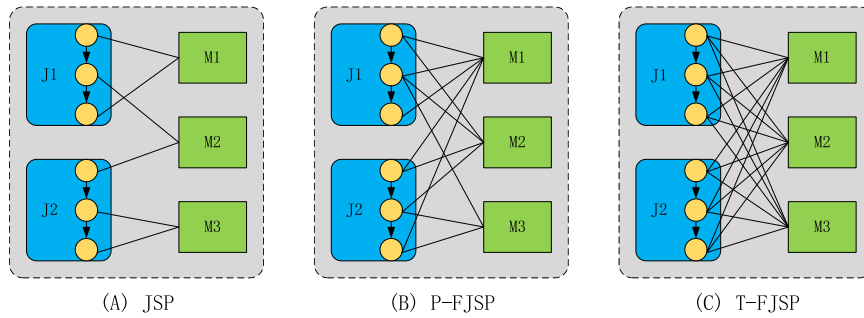


Fig. 1. Relationships among JSP, P-FJSP and T-FJSP.

(4) There are constraints on the machine required for the processing operation.

$$F_{gk} - F_{ik} + M(1 - x_{igk}) \geq P_{gk}, \quad (4)$$

$$i, g = 1, 2, \dots, n; k = 1, 2, \dots, m$$

where n and m denotes the number of jobs and machines respectively. F_{gk} , P_{gk} indicates the completion time and processing time of job g on machine k , M means a sufficiently large positive number. x_{igk} represents the processing priority of job i and job g on machine k . If the job i is processed first, then x_{igk} is 1, otherwise is 0.

(5) One machine cannot process two or more operations at the same time.

$$B_{ijk} \geq F_{gzk} - Z_{ijgzk} \cdot M, \forall k \in M_{ij} \cap M_{gz} \quad (5)$$

$$B_{gzk} \geq F_{ijk} - (1 - Z_{ijgzk}) \cdot M, \forall k \in M_{ij} \cap M_{gz} \quad (6)$$

where Z_{ijgzk} represents whether the j th operation of the i th job O_{ij} is processed before the z th operation of the g th job O_{gz} on machine k , 1 if yes, 0 otherwise. $M_{ij} \cap M_{gz}$ implies the available machine set that can process operation O_{ij} and operation O_{gz} at the same time.

(6) When processing different operations of the same job, it is necessary to follow the specified sequence.

$$B_{ijk} \geq F_{i(j-1)k}, k \in M_{ij}, \quad (7)$$

$$\forall i = 1, 2, \dots, n, \forall j = 2, \dots, K_i$$

where K_i represents the number of operation of job i .

(7) The machine is turned on, and all jobs can be processed at zero time.

$$B_{ijk} \geq 0 \quad (8)$$

$$F_{ijk} \geq 0 \quad (9)$$

(8) Only one machine can be selected for each operation.

$$\sum_{k=1}^m R_{ijk} = 1 \quad (10)$$

where R_{ijk} denotes whether the machine k processes operation O_{ij} , 1 means processed, 0 means unprocessed.

The optimization objective of LSFJSP mainly includes maximum completion time, maximum machine load, processing cost, etc. This paper applies the maximum completion time to test the performance of the algorithm, that can be given by following:

$$f = \min(\text{Makespan}) = \min \max_{1 \leq k \leq m} \left\{ \max_{1 \leq i \leq n} FT_k \right\} \quad (11)$$

where *Makespan* represents the maximum completion time of all machine, it is the main indicator to measure the processing

efficiency. FT_k is expressed as:

$$FT_k = \sum_{i=1}^n \sum_{j=1}^{K_i} \sum_{k=1}^m (B_{ijk}R_{ijk} + P_{ijk}R_{ijk}) \quad (12)$$

2.3. The significance and challenge of LSFJSP

FJSP has received much attention in recent years that many extensions of FJSP such as fuzzy FJSP [10], machine breakdowns FJSP [44], new job arrivals FJSP [45] are proposed. The above researches have overlooked an important issue: with the continuous reform of industrial production, the ordinary FJSP has difficulty meeting urgent industrial needs. The LSFJSP is a development by the FJSP. The constraints and objective functions of LSFJSP are the same as ordinary FJSP. But, the number of jobs and machines in the LSFJSP is far more than in the ordinary FJSP, many methods are difficult to work with. Therefore, to meet the urgent technical needs of the manufacturing industry, it is challenging and necessary to design an effective method to solve LSFJSP.

3. Algorithm preliminaries

3.1. Inspiration of dragonfly algorithm

As we are well known, the dragonfly is a common insect in nature that prey on insects smaller than them. They always form a giant swarm or several small swarms and make constant interactions to prey efficiently. The behaviors and the flight characteristics of dragonflies are related to their predatory purpose so that dragonflies constantly adjust their prey strategy according to the distribution situation of food. In other words, dragonflies try to form more small swarms and search for food in a more extensive area when the food is a shortage. Otherwise, a massive number of dragonflies make migrating in one direction over long distances to prey on food as well as evade enemies. These two swarming behaviors correspond to the two main phases of optimization using MAs: exploration and exploitation. The DA is easy to understand in construction and implementation and possesses super global search accuracy.

3.2. Mathematical formula of dragonfly algorithm

The behaviors of dragonflies consist of separation, alignment, cohesion and attraction towards food, distraction outwards enemy [31]. The characteristics and the mathematical formulas of these five behaviors are represented below.

- Separation behavior refers to the mechanism of the dragonflies avoiding collision with other individuals in the neighborhood that is mathematically described as follows:

$$Q_i = - \sum_{l=1}^0 (X - X_l) \quad (13)$$

where X represents the position of the current individual, X_l represents the position of the l th neighbor, O is the number of neighbors.

- Alignment behavior refers to the tendency of dragonflies to velocity matching according to other surrounding individuals that is mathematically described as follows:

$$K_i = \frac{\sum_{l=1}^O V_l}{O} \quad (14)$$

where V_l represents the velocity of the l th neighbor.

- Cohesion behavior indicates the ability of dragonflies flying towards the center of the mass of the neighborhood that is mathematically described as follows:

$$P_i = \frac{\sum_{l=1}^O X_l}{O} - X \quad (15)$$

- Hunting behavior refers to the mechanism by which dragonflies are attracted to food that is mathematically described as follows:

$$C_i = X^+ - X \quad (16)$$

where X^+ represents the position of optimal food.

- Escape behavior refers to the ability of dragonflies to escape from the enemies that is mathematically described as follows:

$$R_i = X^- + X \quad (17)$$

where X^- represents the position of the enemy.

The above five dragonfly behaviors are the basis for the survival of the dragonfly and the necessary conditions for the DA to solve the problem iteratively. In order to be able to reflect the five behaviors of dragonflies in the algorithm, DA utilizes two vectors to solve the optimization problem: step vector (ΔX) and position vector (X). If there is at least one neighbor around the dragonfly updated during each iteration, update the step vector (ΔX) by the following equation:

$$\Delta X_{t+1} = (qQ_i + kK_i + pP_i + cC_i + rR_i) + g\Delta X_t \quad (18)$$

where q represents separation weight, k represents alignment weight, p represents cohesion weight, c represents food attraction factor, r represents enemy threathation factor, g is the inertia weight. After updating the step vector (ΔX), update the position vector (X) by the following equation:

$$X_{t+1} = X_t + \Delta X \quad (19)$$

Nevertheless, if there is no neighbor around the dragonfly updated, the neighbor effect will be invalid. In order to make the algorithm keep the randomness and have a pretty global search ability, we assume that dragonfly will make stochastic walking that follows the *Le'vy* flight rule. The following equation updates the position vector (X):

$$X_{t+1} = X_t + Le'vy(d) \times X_t \quad (20)$$

where t indicates the number of current iteration and d is the dimension of solution. The mathematical description of *Le'vy* rule is denoted by the following equation:

$$Le'vy(x) = 0.01 \times \frac{t_1 \times \sigma}{|t_2|^{\frac{1}{\beta}}} \quad (21)$$

$$\sigma = \left(\frac{\Gamma(1+\beta) \times \sin(\pi\beta/2)}{\Gamma((1+\beta)/2) \times \beta \times 2^{(\beta-1)/2}} \right)^{1/\beta} \quad (22)$$

where t_1 and t_2 are two random numbers in $[0, 1]$, $\beta = 0.5$, $\Gamma(x) = (x-1)!$.

3.3. Opposition-based learning

Opposition-based learning (OBL) strategy [46] is a popular learning strategy that has been widely adopted as a powerful tool to enhance the performance of algorithms. When calculating a feasible solution X to a given problem, a new chance which makes the candidate solution closer to the best solution will be obtained, if the opposite solution of X is simultaneously computed. The concepts of the opposite number and the opposite point are defined as follows.

Opposite number. Assume X is a real number that in $[A, B]$. Then, the opposite number X^o can be defined as Eq. (23), where A, B is the lower and upper limit, respectively.

$$X^o = A + B - X \quad (23)$$

Opposite point. Suppose $X = (X_1, X_2, \dots, X_D)$ is a point in the D -dimensional space. Then, the multi-dimensions opposite point of X can be defined by the following equation:

$$X_j^o = A_j + B_j - X_j, j = 1, 2, \dots, D \quad (24)$$

A_j and B_j represents the lower and upper limit of the i th dimension respectively, both of which change along with the iteration.

The search space of OBL strategy can be dynamically expanded to the space from the current number to the opposite number for a bigger possibility of being closer to the optimum.

3.4. Dynamic opposite learning

Although the OBL strategy can increase the probability of algorithm closing to the optimal solution, converging the search space to the local optimal position that exists in the space from the current number to the opposite number is inevitable. To this end, the DOL strategy that is proposed by Xu et al. [41] is utilized in this paper to improve the capability of the algorithm to escape the optimal local solution. The DOL strategy makes a slight change based on the OBL strategy by replacing X^o with X^{ro} ($X^{ro} = rand \cdot X^o$, $rand \in [0, 1]$). The symmetric search space can be altered as asymmetric search space which has the characteristic of dynamic adjusting along with a random number.

Assume L represents the lower limit of solution, U represents the upper limit of solution. There are two cases among X^{ro} , L and U , that is $X^{ro} \in [L, U]$ and $X^{ro} \notin [L, U]$. When $X^{ro} \in [L, U]$, there are three relationships among X^{ro} , X^o and X : 1) X^{ro} between X^o and X , 2) X^{ro} is bigger than X^o , 3) X^{ro} is small than X .

If $X^{ro} \notin [L, U]$, $X^{do} = X + rand \cdot (X^{ro} - X)$ may more than the upper limit when $X^{ro} > U$, or $X^{do} = X + rand \cdot (X^{ro} - X)$ may less than the lower limit when $X^{ro} < L$. It is must to regenerate the X^{do} by random rule when $X^{do} \notin [L, U]$.

3.5. The mathematical model of DOL

Before integrating the DOL strategy into the DA, elaborate on the mathematical model of the DOL strategy is necessary.

3.5.1. Dynamic opposite number

The dynamic opposite number is utilized to describe the procedure of dynamic opposite learning. Assume L, U represents lower limit and upper limit respectively, X is real number that in $[L, U]$. w implies learning weight, S_1, S_2 are two random numbers in $[0, 1]$. Then, the dynamic opposite number X^{do} can be calculated as follows:

$$X^{do} = X + w \cdot S_1 \cdot (S_2 \cdot X^o - X) \quad (25)$$

$$X^o = U + L - X \quad (26)$$

3.5.2. Dynamic opposite point

The dynamic opposite point is the extension of dynamic opposite number in the multidimensional space. Assume exists a D-dimensional space that including several feasible solutions. $X = \{X_1, X_2, \dots, X_D\}$ represents the current solution that is restricted by the lower limit: $L = \{L_1, L_2, \dots, L_D\}$ and upper limit: $U = \{U_1, U_2, \dots, U_D\}$, S_3, S_4 are two random numbers in $[0, 1]$. The dynamic opposite point of X can be represented as follows:

$$X_j^{do} = X_j + w \cdot S_3 \cdot (S_4 \cdot X_j^o - X_j), j = 1, 2, \dots, D \quad (27)$$

$$X_j^o = U_j + L_j - X_j, j = 1, 2, \dots, D \quad (28)$$

3.5.3. DOL based optimization

The DOL strategy will be integrated into the algorithm's initialization phase and the iteration phase. In the initialization phase, generate the population F^{DO} by the population F and regard F^{DO} and F as a whole population $F^{DO} \cup F$, then calculate the fitnesses of the $F^{DO} \cup F$ and select the first half best individuals from $F^{DO} \cup F$ to the population F^{sort} . In the iteration phase, if jumping action satisfies, generate F^{DO} by F^{sort} and regard F^{DO} and F^{sort} as a whole $F^{DO} \cup F^{sort}$, then calculate the fitnesses of $F^{DO} \cup F^{sort}$ and select the first half best individuals from $F^{DO} \cup F^{sort}$ to F^{sort} . Regenerate individual while the member of F , F^{DO} and F^{sort} beyond the boundary.

4. DOL based DA

The above have discussed the principles of the dragonfly algorithm, characteristics and mathematical formulas of the DOL strategy. In this section, a variant of DA named DOLDA is formulated to tackle the precocity that always puzzles many optimization algorithms.

4.1. DOL population initialization

In terms of the essence of DOLDA, that is selecting first half optimal solutions from the current solution and its opposite solutions. The initialization of the DOL population is supported by DA population initialization. Corresponding initialization rules are described as follows:

$$F_{ij}^{DO} = F_{ij} + R1_i \cdot (R2_i \cdot (U_j + L_j - F_{ij}) - F_{ij}) \quad (29)$$

where F_{ij} represents the j th dimension of i th individual in population generated by DA, $R1_i$ and $R2_i$ are two random numbers in $[0, 1]$, U_j and L_j respectively represents the upper limit and lower limit of j th dimension of i th individual, F_{ij}^{DO} represents the opposite solution of F_{ij} . The boundary of F_{ij}^{DO} should be checked under the rule (30) during the entire DOL population initialization stage.

$$F_{ij}^{DO} = Reg, \text{ if } F_{ij}^{DO} < L_j \parallel F_{ij}^{DO} > U_j \quad (30)$$

where Reg denotes the random number in $[L_j, U_j]$.

4.2. DOL generation jumping

DOL generation jumping is similar to DOL population initialization but can be applied in the entire iteration to help algorithm to escape local optima. To trigger DOL generation jumping reasonably, DOL strategy applies a jumping rate factor denoted by $\vartheta \in [0, 1]$ to represent the probability of the algorithm execute the jumping action. Suppose the random number generated from 0 to 1 is less than the jumping rate factor ϑ , DOL generation jumping is content and makes jumping action. The jumping action is described as follows:

$$F_{ij}^{DO} = F_{ij} + w \cdot R3_i \cdot (R4_i \cdot (U_j + L_j - F_{ij}) - F_{ij}) \quad (31)$$

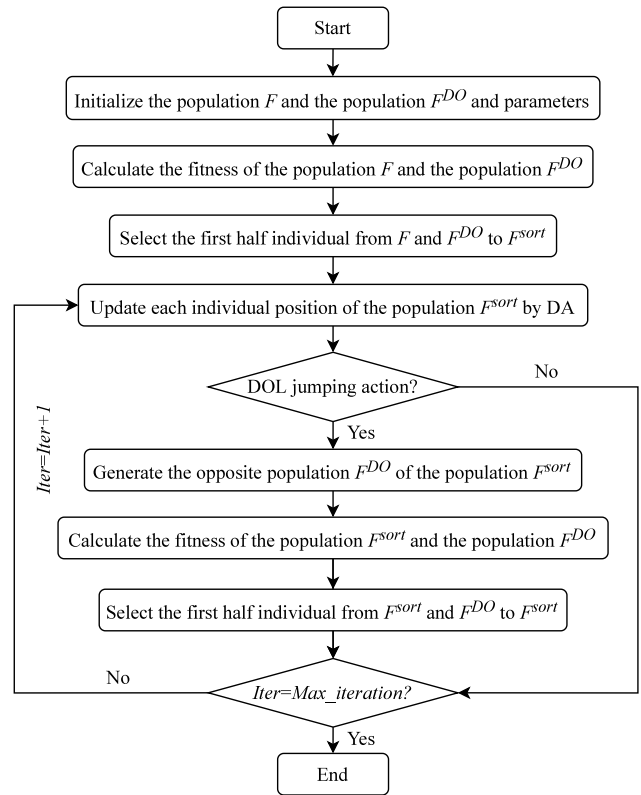


Fig. 2. The flowchart of the DOLDA.

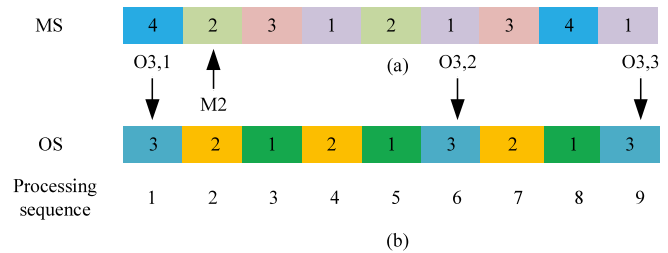


Fig. 3. An example of coded solution by previous way.

where w represents the learning weight that can be defined differently according to different scenes and conditions, $R3_i$ and $R4_i$ are two random numbers in $[0, 1]$. A boundary check is necessary and corresponding rules are the same as the DOL population initialization stage. There is hardly a difference between the population initialization and the generation jumping of DOL, as the DOL generation jumping stage is the extension and continuation of the DOL population initialization stage.

4.3. DOLDA procedure

As mentioned previously, DOLDA is the variant of DA that DOL strategy is embedded in the DA. The DA procedure and the DOL strategy is introduced before. In this subsection, We summary the steps of DOLDA and draw the flowchart of DOLDA in Fig. 2 ($Iter$ records the number of iterations, $Max_iteration$ represents the maximum number of iterations). Put the pseudo-code of DOLDA into Algorithm 1 (N means the population size, D indicates the dimension).

The optimization course of DOLDA consists of two stages: population initialization stage and generation jumping stage. For the population initialization stage, generate the initial population F and the opposite population F^{DO} , then calculate the fitness of each individual and select the top N best individuals into F^{sort} . For the generation jumping stage, generate the opposite population F^{DO} of F^{sort} by Eq. (31), then select the top N best individuals from the opposite population F^{DO} and F^{sort} to F^{sort} when jumping rate ϑ is minor than the random number in $[0, 1]$.

Algorithm 1: The DOLDA

```

1: Generate the initial population;
2: For  $i=1:N$ 
3:    $R1_i = rand(0,1)$ ,  $R2_i = rand(0,1)$ ;
4:   For  $j=1:D$ 
5:      $F_{ij}^{DO} = F_{ij} + R1_i \cdot (R2_i \cdot (U + L_j - F_{ij}) - F_{ij})$ ;
6:     Check the boundaries;
7:   End for
8: End for
9: Select the top  $N$  best individuals from  $F \cup F^{DO}$  to  $F^{sort}$ ;
10: Set  $Iter = 0$ ;
11: While  $Iter < Max\_iteration$ 
12:   Calculate the fitness of all individual;
13:   For  $i=1:N$ 
14:     Calculate the  $Q, K, P, C, R$  using Eq. (13) to Eq. (17);
15:     If  $i$ th individual has at least one neighbor
16:       Update the velocity vector using Eq. (18);
17:       Update the position vector using Eq. (19);
18:     Else
19:       Update directly the position vector using Eq. (20);
20:     End if
21:     Check the boundaries;
22:   End for
23:    $Iter++$ ;
24:   For  $i=1:N$ 
25:     If  $rand < \vartheta$ 
26:        $R3_i = rand(0,1)$ ,  $R4_i = rand(0,1)$ ;
27:       For  $j=1:D$ 
28:          $F_{ij}^{DO} = F_{ij}^{sort} + w \cdot R3_i \cdot (R4_i \cdot (U_j + L_j - F_{ij}) - F_{ij}^{sort})$ ;
29:         Check the boundaries;
30:       End for
31:     End if
32:   End for
33:   Select top  $N$  best individuals from  $F^{sort} \cup F^{DO}$  to  $F^{sort}$ ;
34: End while

```

4.4. Decoding and encoding

There are several schemes to decode and encode FJSP, and the popular is the use of Machine Selection (MS) and Operation Sequence (OS) vectors proposed by Gao et al. to perform chromosome representation [47]. In Fig. 3, the MS vector represents the machine used by each operation of each job, and the OS vector denotes the execution order of each operation of each job. For instance, the second element “2” in the MS vector represents the first operation of the second job adopts machine two for its processing. The same color box represents the operation of the same job in the OS vector. Such as the first dark blue box “3” represents the first operation of the third job is processed firstly, the second dark blue box “3” represents the second operation of the third job is processed by the sixth, the third dark blue box “3” represents the third operation of the third job is processed

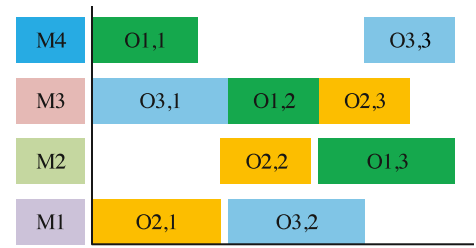


Fig. 4. The Gantt chart after decoding in previous way.

by the ninth. In this way, we can get the start and finish time of each operation of each job and the machines they use. The coded solution can be decoded to a Gantt chart in previous ways are as shown in Fig. 4.

The above representation of solutions will increase the usage of memory. Learn from the representation of the chromosome in literature [48], this paper introduces a $X \times Y$ matrix (X is equal to the number of jobs, Y is equal to the number of operations) to represent the chromosomes and design a novel decoding way to map chromosomes to the Gantt chart. An illustration of encoding and decoding by our way is shown in Fig. 5.

The decoding process of the MFJS1 that comes from Fadatat [49] is shown in Fig. 5. A 5×3 matrix is used to represent the feasible solution. The element in the i th row and j th column of the matrix represents the number of machines used in the j th operation of the i th job. The transport time is not considered throughout the entire process. The decoding principles of the each operation are as followed.

For the first operation, we firstly look for the jobs processed by machine one, and that is the third job and the fifth job. We let the top-ranked job be processed firstly. Machine one immediately processes the fifth job when it finishes the third job. Thus, the start time of the third job is 0 and its finish time is 87; the start time of the fifth job is 87 and its finish time is 215. Then, We look for the job processed by machine two, and that is the fourth job, due to machine two only process the fourth job, so the start time of the fourth job is 0 and its finish time is 65. Next, We look for the job processed by machine three, and that is the first job and the second job. Similar to the first two steps, we can get the start time and the finish time of the first job one are 0 and 145; the start time and the finish time of the second job are 145 and 295. There is no job to be processed by machine 4–6. The machine traversal ends.

The decoding principle of the second operation is the same as that of the first operation. However, not all machines can be directly used in the second operation. Therefore, before we traverse each machine to find the start time and the finish time of the second operation, it is necessary to calculate the maximum finish time of each machine on the first operation and use Eq. (2) to determine the starting time of the next operation. Similarly, the decoding principle of the third and subsequent operation is the same as the second operation.

5. Result and discussion

5.1. Experiment data set

To better verify the performance of DOLDA, extensive numerical experiments on the CEC 2013 and CEC 2014 test functions are performed [50,51]. These test functions are classified as unimodal functions, multimodal functions, and hybrid functions. Unimodal functions are utilized to test the exploitation capability of DOLDA

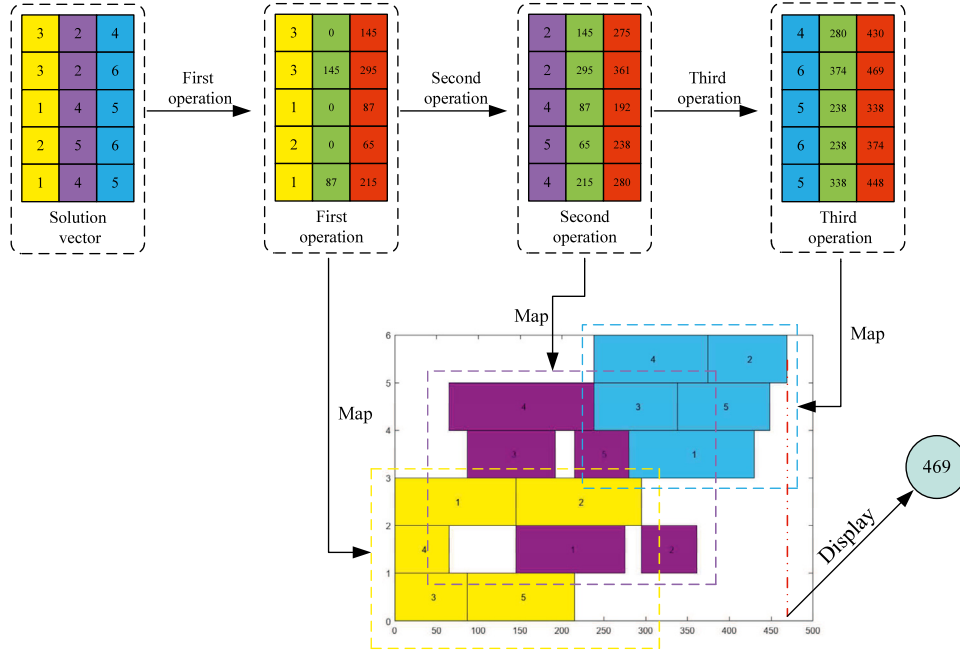


Fig. 5. The Gantt chart after decoding in our way.

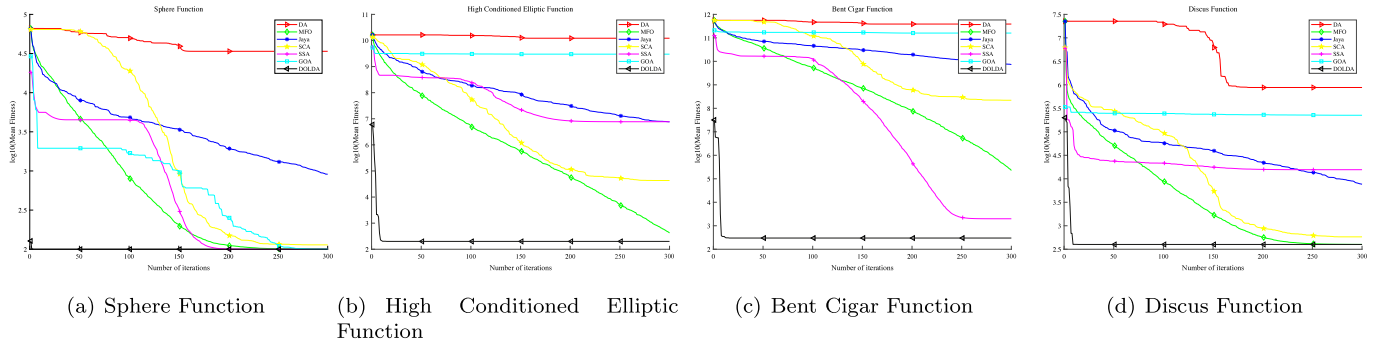


Fig. 6. The convergence curves of the unimodal functions.

because there is no local optima among them. Multimodal functions are utilized to test the exploration capability of DOLDA because they have many local optima. Comprehensively, we utilize the hybrid functions to test the ability of exploitation and exploration. The details of these functions are shown in Table 1.

These first 20 functions presented in Table 1 are the minimum functions. We add offset 100 to function F1, add offset 200 to function F2, and so on. The hybrid functions are composed of the functions that are from the first 20 functions. We do not add offset into the hybrid functions, and their optimal values are 0. The following equation describes the hybrid functions:

$$f(x) = \sum_{i=1}^n \{\varphi_i \cdot [\theta_i \cdot g_i(x) + bias_i]\} + f^* \quad (32)$$

In the above equation, $f(x)$ is new composition function, $g_i(x)$ represents the i th basic function used to construct the composition functions, θ_i represents the value that used to control each $g_i(x)$'s height, n represents the number of basic functions, $bias_i$ defines which optima is global optimum, f^* represents the offset

of function. φ_i is calculated by the following equation:

$$w_i = \frac{1}{\sqrt{\sum_{j=1}^D (x_j - o_{ij})^2}} \exp\left(-\frac{\sum_{j=1}^D (x_j - o_{ij})^2}{2D\sigma_i^2}\right) \quad (33)$$

$$\varphi_i = w_i / \sum_{i=1}^n w_i \quad (34)$$

In Eq. (33), o_i represents the new shifted optimal position for each $g_i(x)$ that defines the global and local optimal position, σ_i is used to control each $g_i(x)$'s coverage range, a small σ_i gives a narrow range for that $g_i(x)$, D denotes the dimension of problem. The component information of hybrid functions is shown in Table 2.

5.2. Parameter settings

This paper introduces six other classic meta-heuristic algorithms that including dragonfly algorithm (DA) [52], moth-flame optimization algorithm (MFO) [53], Jaya algorithm [54], sine cosine algorithm (SCA) [55], salp swarm algorithm (SSA) [56],

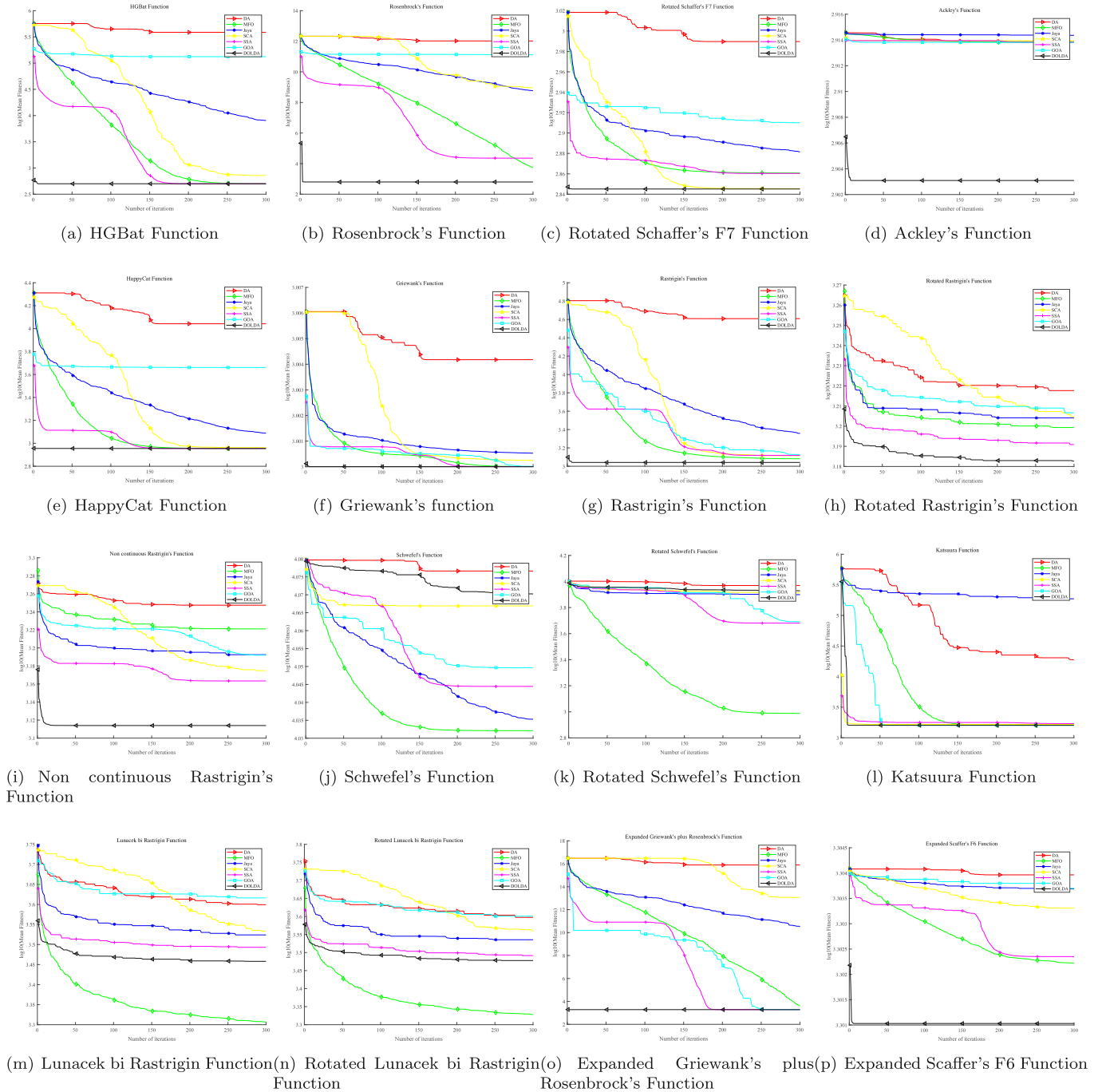


Fig. 7. The convergence curves of the multi-modal functions.

grasshopper optimization algorithm (GOA) [57] as comparison algorithm to examine the merit of the DOLDA. For all algorithm, the population size and number of iterations is set as 100 and 300 respectively. In detail, the parameter settings of each algorithm refer to the suggested values of the original paper. The learning weight w of DOL strategy is set as 1. The parameter jumping rate ϑ is analyzed in the following subsection.

5.3. Analysis of jumping rate ϑ

In the DOLDA, the jumping rate ϑ is an important parameter influencing the algorithm's results. So, it is inevitable to discuss that what is the most suitable jumping rate. For this purpose, this subsection will discuss what is the most suitable jump rate ϑ for DOLDA. For the rigors of the experiment, jumping rate ϑ is divided from 0 to 1 into ten levels: [0.1, 0.2, ..., 1]. Ten levels of

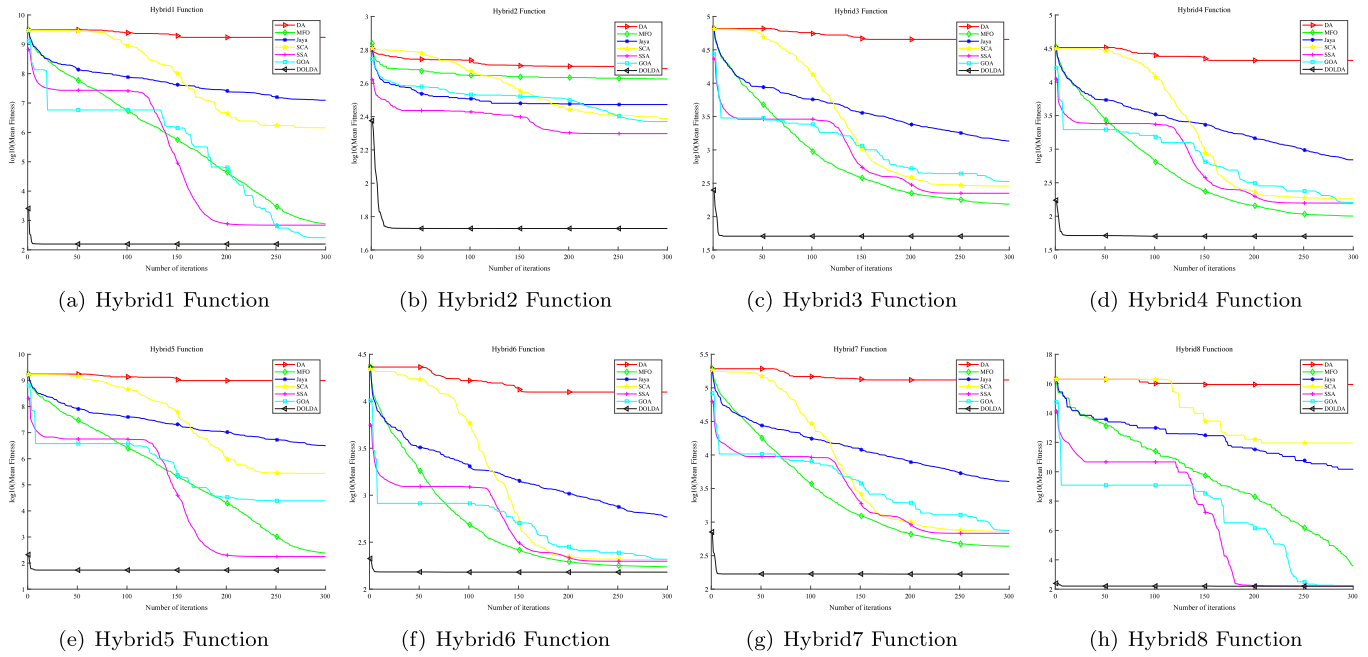


Fig. 8. The convergence curves of the hybrid functions.

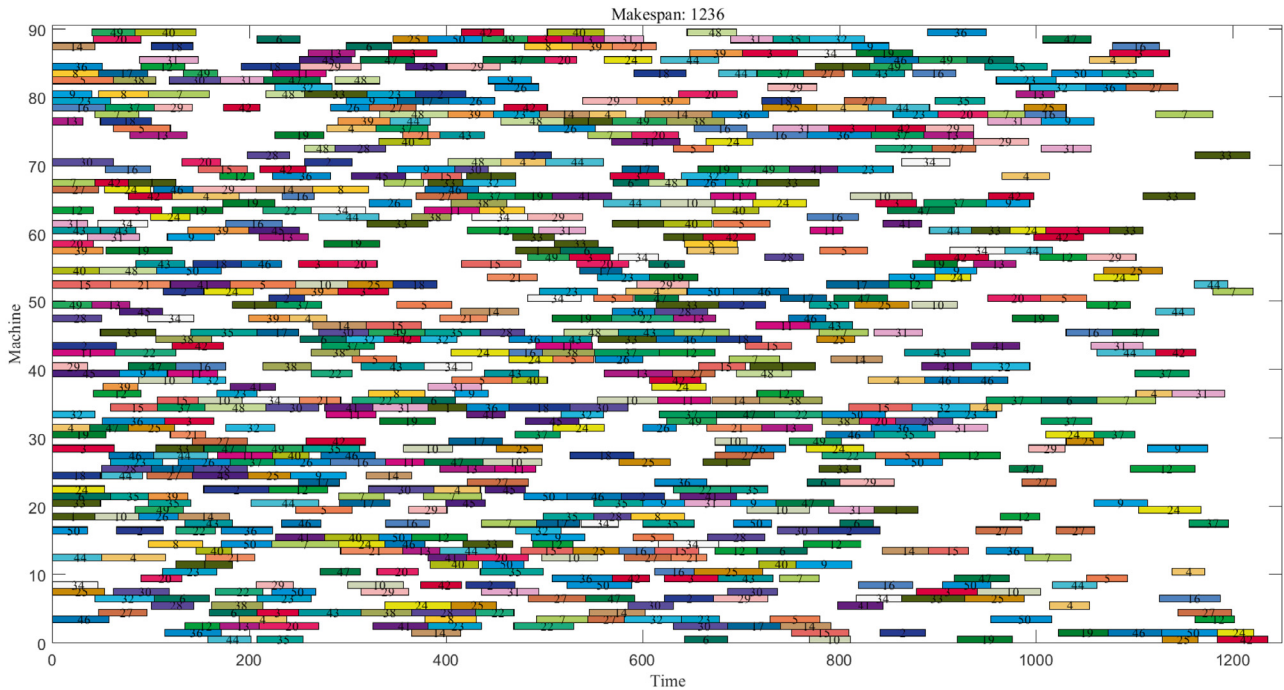


Fig. 9. The Gantt chart of Instance.1 by DOLDA.

jump parameters are run ten times on all 28 test functions. A few representative results are shown in Table 3.

The experimental results show that the optimization is the best when the jumping rate ϑ is set to 0.5, followed by 0.2, 0.3 and 0.6. Therefore, the jumping rate ϑ is set to 0.5 in the subsequent algorithm comparison tests.

5.4. Analysis of test functions

This paper performs ten counts of experiments for each test function and collects the Mean and Standard Deviation (Std) value of the experimental results into the three tables. The optimization results of the unimodal, multimodal, and hybrid functions

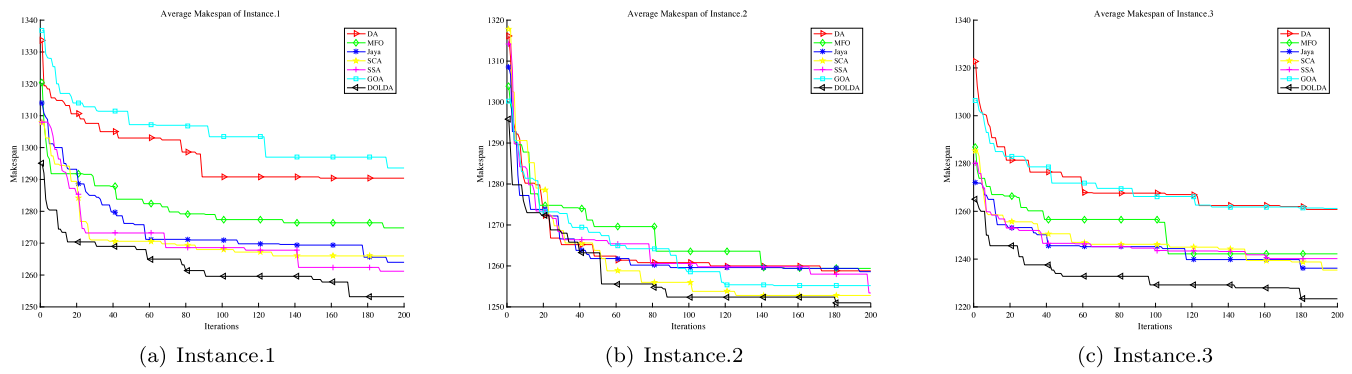


Fig. 10. The mean convergence curve of Instance.1-Instance.3.

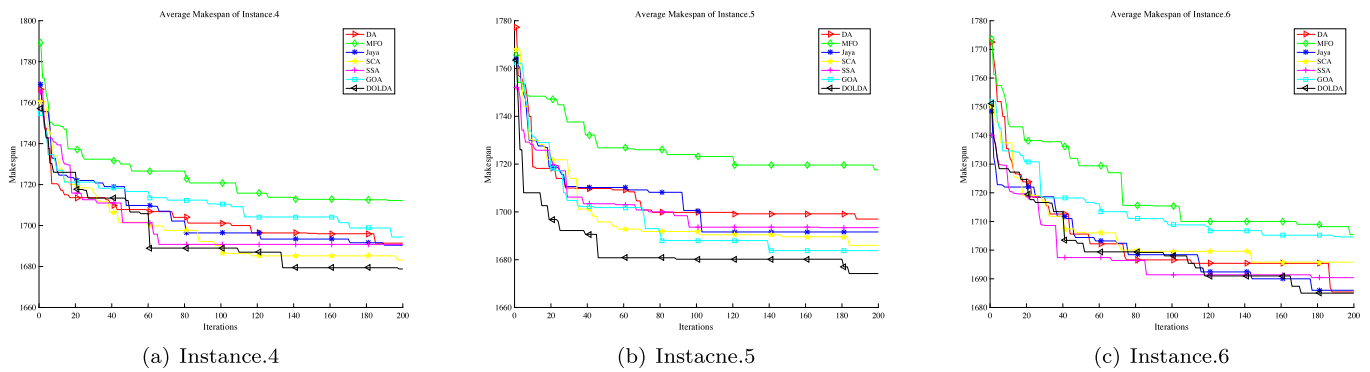


Fig. 11. The mean convergence curve of Instance.4-Instance.6.

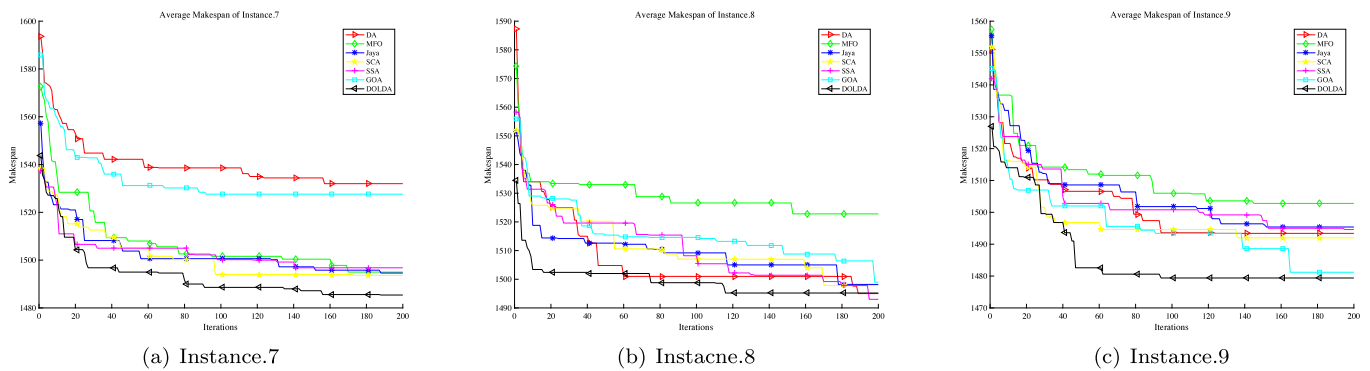


Fig. 12. The mean convergence curve of Instance.7-Instance.9.

is shown in Tables 4–6, respectively. The bold results with * indicate the optimal results. For Mean value, the DOLDA can obtain optimal on all test functions besides the F14, F15, F17, F18. Moreover, about Std value, the results of DOLDA are smaller than other algorithms on almost test functions, which represents the optimization of DOLDA is more stable and precise than other algorithms. In general, DOLDA can outperform other compared algorithms in most cases.

5.5. T-test

Due to the small size samples are included in our experiments, it is necessary to utilize the Two Independent-Samples T-test to

test whether there is an accidental similarity between DOLDA and other contrasted algorithms. We choose 0.05 as the level of significance. The hypothesis is accepted when $P > 0.05$, the test results indicate no significant difference between the two compared algorithms. The hypothesis is rejected when $P \leq 0.05$, the test results indicate significant differences between the DOLDA and contrasted algorithms. The test results are shown in Table 7. We denote the value of p more than 0.05 as “+” and denote the value of p less than 0.05 as “-”. ‘Same’ denotes the total number of DOLDA has no significant difference with other algorithms, ‘Better’ represents the total number of DOLDA that has significant differences with other algorithms.

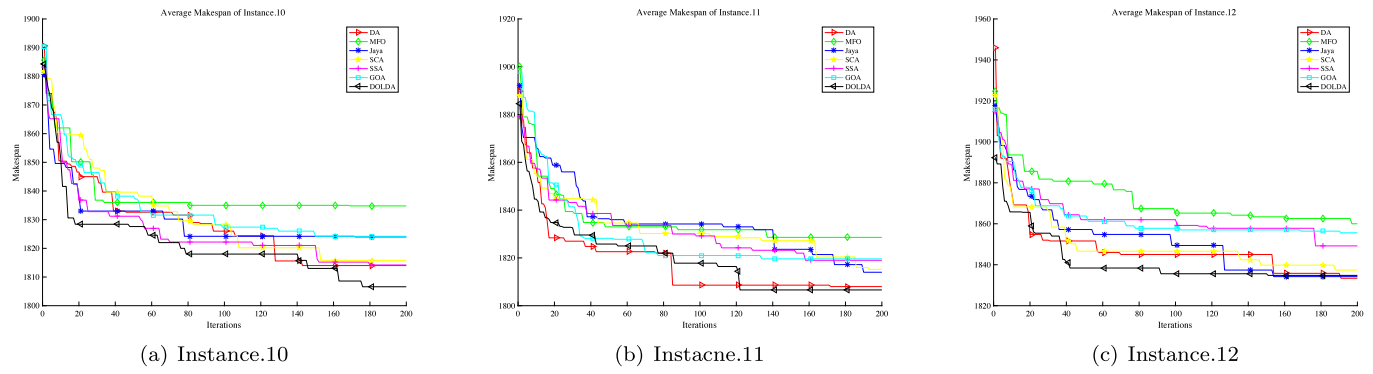


Fig. 13. The mean convergence curve of Instance.10-Instance.12.

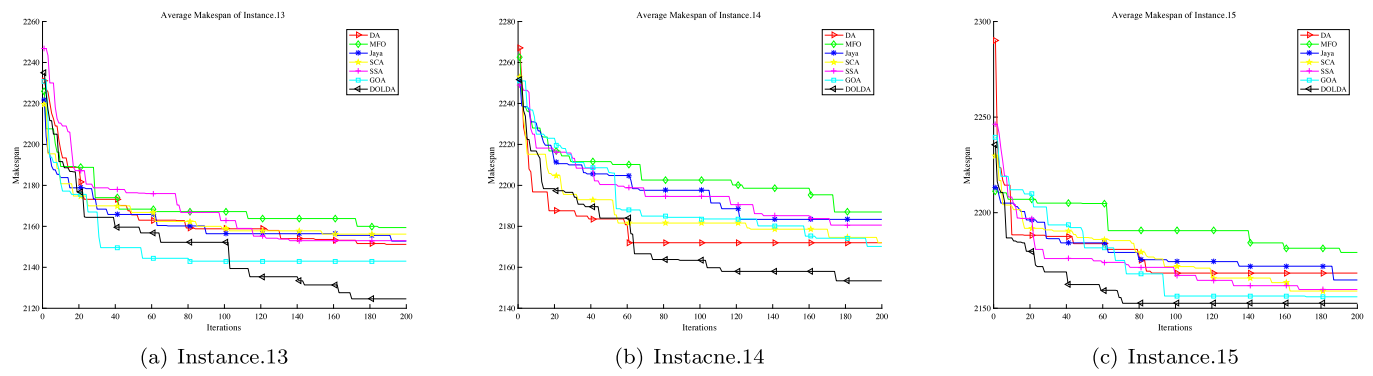


Fig. 14. The mean convergence curve of Instance.13-Instance.15.

Table 1

Test function details.

Label	Functions	Dims	Optimum	Properties
F1	Sphere Function	30	100	Unimodal
F2	High Conditioned Elliptic Function	30	200	Unimodal
F3	Bent Cigar Function	30	300	Unimodal
F4	Discus Function	30	400	Unimodal
F5	HGBat Function	30	500	Multimodal
F6	Rosenbrock's Function	30	600	Multimodal
F7	Rotated Schaffer's F7 Function	30	700	Multimodal
F8	Ackley's Function	30	800	Multimodal
F9	HappyCat Function	30	900	Multimodal
F10	Griewank's Function	30	1000	Multimodal
F11	Rastrigin's Function	30	1100	Multimodal
F12	Rotated Rastrigin's Function	30	1200	Multimodal
F13	Non continuous Rastrigin's Function	30	1300	Multimodal
F14	Schwefel's Function	30	1400	Multimodal
F15	Rotated Schwefel's Function	30	1500	Multimodal
F16	Katsuura Function	30	1600	Multimodal
F17	Lunacek bi-Rastrigin Function	30	1700	Multimodal
F18	Rotated Lunacek bi-Rastrigin Function	30	1800	Multimodal
F19	Expanded Griewank's plus Rosenbrocks Function	30	1900	Multimodal
F20	Expanded Scaffer's F6 Function	30	2000	Multimodal
F21	Hybrid Function.1	30	0	Hybrid
F22	Hybrid Function.2	30	0	Hybrid
F23	Hybrid Function.3	30	0	Hybrid
F24	Hybrid Function.4	30	0	Hybrid
F25	Hybrid Function.5	30	0	Hybrid
F26	Hybrid Function.6	30	0	Hybrid
F27	Hybrid Function.7	30	0	Hybrid
F28	Hybrid Function.8	30	0	Hybrid

Table 2
The component information of hybrid function.

Hybrid function	σ	θ	bias	Basic function
F21	[10, 20, 30, 40, 50]	[1, 1e−6, 1e−26, 1e−6, 0.1]	[0, 100, 200, 300, 400]	[F6,F5,F3,F4,F1]
F22	[20, 20, 20]	[1,1,1]	[0,100,200]	[F13,F13,F13]
F23	[20, 20, 20]	[1,1,1]	[0,100,200]	[F12,F12,F12]
F24	[20, 20, 20]	[0.25,1,2.5]	[0,100,200]	[F9,F12,F5]
F25	[10, 30, 50]	[0.25,1,2.5]	[0,100,200]	[F6,F7,F8]
F26	[10, 10, 10, 10, 10]	[0.25, 1, 1e−7, 2.5, 10]	[0, 100, 200, 300, 400]	[F5,F12,F2,F9,F10]
F27	[10, 10, 10, 10, 10]	[100, 10, 2.5, 25, 0.1]	[0, 100, 200, 300, 400]	[F10,F12,F11,F9,F1]
F28	[10, 20, 30, 40, 50]	[2.5, 2.5e−3, 2.5, 5e−4,0.1]	[0, 100, 200, 300, 400]	[F1,F7,F19,F6,F15]

Table 3
The sensitivity analysis of jumping rate ϑ .

Jump rating	F9	F12	F14	F15	F17	F18
$\vartheta = 0.1$	901.77	1530.80	11754.27	8497.54	2917.75	3076.06
$\vartheta = 0.2$	910.88	1522.23*	11687.32	8334.16	2954.03	3010.97
$\vartheta = 0.3$	910.89	1539.03	11804.58	8213.46*	2917.78	3046.07
$\vartheta = 0.4$	901.90	1526.48	11846.15	8295.21	2899.89	3002.75
$\vartheta = 0.5$	901.70*	1522.62	11846.54	8633.41	2869.46*	3011.41
$\vartheta = 0.6$	901.78	1523.50	11680.14*	8466.69	2955.52	3020.49
$\vartheta = 0.7$	901.79	1525.41	11835.30	8394.92	2914.03	2993.27
$\vartheta = 0.8$	901.72	1531.67	11808.21	8569.30	2895.06	3035.62
$\vartheta = 0.9$	901.73	1528.10	11792.98	8623.68	2890.57	3015.44
$\vartheta = 1.0$	901.77	1537.04	11779.85	8535.19	2916.96	2976.09*
Jump rating	F19	F23	F25	F27	F28	Best num
$\vartheta = 0.1$	1913.77	50.43	53.68	167.75*	158.87	1
$\vartheta = 0.2$	1913.78	50.36*	53.63	167.75*	158.99	3
$\vartheta = 0.3$	1913.32	50.36*	53.62*	167.77	158.87	3
$\vartheta = 0.4$	1913.78	50.36*	53.66	167.76	158.87	1
$\vartheta = 0.5$	1911.77*	50.36*	53.63	167.78	158.83*	5*
$\vartheta = 0.6$	1913.77	50.36*	53.62*	167.79	159.21	3
$\vartheta = 0.7$	1917.57	50.36*	53.65	167.82	159.05	1
$\vartheta = 0.8$	1913.78	50.36*	53.72	167.96	159.15	1
$\vartheta = 0.9$	1913.78	50.36*	53.67	167.82	159.25	1
$\vartheta = 1.0$	1913.78	50.37	53.63	168.56	159.99	1

Best num represents the number of the best solution.

From Table 7, we find that DOLDA and other algorithms are significantly different in almost all test functions. Among the compared algorithms, MFO is the closest to DOLDA, but there are still many differences between both. In general, we can conclude that DOLDA is significantly excellent from other algorithms.

5.6. Convergence analysis

The convergence trends of all algorithms on test functions are shown in Figs. 6–8. It can be seen that DOLDA holds the lower iteration initial point due to the addition of the DOL strategy. DOLDA obtains the best solving results among all test functions except F14, F15, F17, and F18. On all unimodal functions, the iteration curve of DOLDA converges faster and is always at the bottom. As a result, DOLDA shows a more robust global search capability compared to other algorithms. On multimodal function F5, F7, F9–F11, F19, DOLDA occupies a competitive achievement for reaching the optima at the beginning of the iteration. On multimodal functions F6, F8, F13, F16, F20, DOLDA can also obtain the optima as the behavior of the abrupt change at the beginning of the iteration. Thus, we can conclude that DOLDA owns better local search competence than other compared algorithms. The DOLDA can always get the best optimization on all hybrid functions because of lower initial iteration points or abrupt changes at the beginning of the iteration. It shows that the DOLDA has better

global and local search capabilities and more robust balance capabilities between exploration and exploitation. DOL strategy can perfectly balance the algorithm's exploitation and exploration process to significantly enhance DA's search capabilities by changing the search space asymmetrically.

5.7. Application to the LSFJSP

Since it is difficult to find LSFJSP instances to verify our proposed algorithm, in this paper, we design 15 LSFJSP instances by Brandimarte rule [58] to evaluate further and contrast the implementation of DOLDA and other contrasted algorithms. The details of LSFJSP instances are placed in Table 8.

In the original Brandimarte rule, the processing time of all operations is within a range, the processing time of the same operation on different machines may vary significantly. To this end, this paper designs the *basic_time* that is restrict by the *aproc* and *offset_time* that is restrict by the *mot* to reduce this difference. The processing time of the operation O_{ij} on the machine k can be calculated as:

$$P_{ijk} = \text{basic_time}_{ij} + \text{offset_time}_k \quad (35)$$

where *basic_time_{ij}* is the basic processing time of O_{ij} , *offset_time_k* is the offset time on machine k .

About parameter settings, we set the number of iterations to 200 and population size to 100. The compared algorithms and the parameter settings of algorithms both are the same as the above test function experiments. The algorithms terminate when the number of iteration values reaches the maximum. We perform ten counts of experiments for each instance. Fig. 9 show the Gantt chart of Instance.1 obtained by DOLDA. The mean convergence curve of each instance is shown in Figs. 10–14. The information of makespan is reported in Tables 9 and 10.

In Table 9, SM and AM mean the smallest makespan and the average makespan in the population. In Table 10, the relative deviation criterion represented by (SD, AD) is employed for the comparing, which is defined as:

$$SD_i = \sum_{j=1}^n (SM_i - SM_j) / SM_i \quad (36)$$

$$AD_i = \sum_{j=1}^n (AM_i - AM_j) / AM_i \quad (37)$$

where i means the current algorithm, j means the compared algorithm, n implies the number of comparison algorithms, SD_i and AD_i respectively represents the sum of the deviations of the smallest makespan and average makespan of the current algorithm and other comparison algorithms. The smaller SD_i and AD_i , the better the effect of the algorithm.

Table 4

The Mean and Std value of the final result of the unimodal function.

Algorithm	F1		F2		F3		F4	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	3.38E+04	1.72E+04	1.22E+10	2.38E+09	3.87E+11	6.17E+10	8.78E+05	2.73E+05
MFO	1.00E+02*	2.68E−02	4.19E+02	1.15E+02	2.25E+05	1.87E+05	4.01E+02	4.30E−01
Jaya	9.08E+02	2.84E+02	7.60E+06	2.19E+06	7.34E+09	3.90E+09	7.63E+03	1.57E+03
SCA	1.14E+02	1.23E+01	4.31E+04	3.60E+04	2.21E+08	3.23E+08	5.81E+02	2.00E+02
SSA	1.00E+02*	2.41E−07	7.69E+06	5.93E+06	1.99E+03	1.24E+03	1.56E+04	3.86E+03
GOA	1.00E+02*	0.00E+00	2.99E+09	5.03E−07	1.60E+11	3.22E−05	2.25E+05	6.14E−11
DOLDA	1.00E+02*	0.00E+00	2.00E+02*	0.00E+00	3.00E+02*	0.00E+00	4.00E+02*	0.00E+00

Table 5

The Mean and Std value of the final result of the multi-modal function.

Algorithm	F5		F6		F7		F8	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	3.87E+05	8.60E+04	1.05E+12	4.40E+11	9.77E+02	3.59E+01	8.20E+02	1.16E−01
MFO	5.01E+02*	2.87E−01	5.27E+03	6.58E+03	7.26E+02	1.31E+01	8.20E+02	1.18E−02
Jaya	8.05E+03	3.40E+03	5.73E+08	3.04E+08	7.61E+02	1.25E+01	8.21E+02	6.85E−02
SCA	7.20E+02	2.14E+02	9.20E+08	2.20E+09	7.00E+02*	3.29E−01	8.20E+02	6.45E−02
SSA	5.01E+02*	2.74E−01	2.27E+04	4.97E+04	7.25E+02	5.43E+00	8.20E+02	1.51E−07
GOA	1.33E+05	3.07E−11	1.32E+11	3.22E−05	8.13E+02	1.20E−13	8.20E+02	0.00E+00
DOLDA	5.01E+02*	0.00E+00	6.29E+02*	1.50E−02	7.00E+02*	0.00E+00	8.00E+02*	0.00E+00
Algorithm	F9		F10		F11		F12	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	1.10E+04	3.64E+03	1.01E+03	2.06E+00	4.06E+04	8.27E+03	1.65E+03	2.36E+01
MFO	9.01E+02*	1.08E−01	1.00E+03*	8.56E−03	1.21E+03	1.73E+01	1.58E+03	1.31E+01
Jaya	1.22E+03	1.22E+02	1.00E+03*	7.57E−02	2.27E+03	2.52E+02	1.60E+03	2.07E+01
SCA	9.18E+02	1.55E+01	1.00E+03*	2.00E−01	1.30E+03	5.35E+01	1.60E+03	2.74E+01
SSA	9.01E+02*	9.69E−02	1.00E+03*	1.23E−02	1.31E+03	5.72E+01	1.55E+03	1.44E+01
GOA	4.58E+03	9.59E−13	1.00E+03*	1.20E−13	1.34E+03	0.00E+00	1.61E+03	5.49E+01
DOLDA	9.01E+02*	1.41E−01	1.00E+03*	0.00E+00	1.10E+03*	0.00E+00	1.52E+03*	1.28E+01
Algorithm	F13		F14		F15		F16	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	1.77E+03	2.82E+01	1.19E+04	6.78E+01	9.31E+03	4.74E+02	1.88E+04	3.04E+04
MFO	1.66E+03	2.69E+01	1.08E+04*	5.29E+01	1.69E+03*	3.15E+02	1.60E+03*	1.59E+00
Jaya	1.56E+03	2.13E+01	1.08E+04*	1.23E+02	7.93E+03	3.55E+02	1.87E+05	3.85E+04
SCA	1.50E+03	5.69E+01	1.17E+04	5.33E+01	8.18E+03	1.95E+02	1.65E+03	3.44E+01
SSA	1.46E+03	2.69E+01	1.11E+04	1.03E+02	4.79E+03	7.20E+02	1.70E+03	6.31E+01
GOA	1.56E+03	7.31E+01	1.12E+04	0.00E+00	4.92E+03	9.59E−13	1.60E+03*	0.00E+00
DOLDA	1.30E+03*	1.42E−04	1.18E+04	1.11E+02	8.63E+03	4.57E+02	1.60E+03*	0.00E+00
Algorithm	F17		F18		F19		F20	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	3.98E+03	1.92E+02	3.96E+03	1.77E+02	7.84E+15	5.15E+15	2.01E+03	3.52E−01
MFO	2.03E+03*	5.25E+01	2.13E+03*	5.95E+01	3.93E+03	3.35E+03	2.01E+03	1.17E+00
Jaya	3.34E+03	1.44E+02	3.43E+03	1.16E+02	3.19E+10	3.71E+10	2.01E+03	3.07E−01
SCA	3.42E+03	1.92E+02	3.63E+03	1.71E+02	1.16E+13	1.62E+13	2.01E+03	7.81E−01
SSA	3.11E+03	1.25E+02	3.10E+03	5.55E+01	1.91E+03*	1.89E+00	2.01E+03	8.10E−01
GOA	4.13E+03	5.16E+02	3.99E+03	5.51E+02	1.92E+03	0.00E+00	2.01E+03	4.79E−13
DOLDA	2.87E+03	8.32E+01	3.01E+03	1.00E+02	1.91E+03*	1.10E−02	2.00E+03*	0.00E+00

From the convergence curves, it is not difficult to find that the convergence curve of DOLDA stays below the convergence curve of other algorithms in all instances except Instance.8 and Instance.12. Especially in Instance.1, 3, 5, 7, 15, DOLDA is in the lead almost at all times. The DOLDA declines sharply in the middle of the iteration in Instance.2, 4, 9, 13, 14. The DOLDA takes the lead in the later iterations in Instance.6, 10, 11.

Table 9 shows that the DOLDA can reach 9 best in SM and 13 best in AM, far surpassing other algorithms. Moreover, in Table 10, DOLDA can also obtain 9 best in SD and 13 best in AD, surpassing other algorithms. For Instance.11, the DOLDA comprehensively leading other algorithms by 20% in SM and for

Instance.1, 3, the DOLDA comprehensively leading other algorithms by 11% in AM. As the foundation of DOLDA, DA shows many advantages compared to the other five comparison algorithms. This not only demonstrates the value of combining DOL strategy with DA, but also provides a powerful framework for the subsequent development of DA.

In short, DA has the inherent ability to solve LSFJSP. The addition of the DOL strategy makes DOLDA more effective search ability than DA. In the face of the high-dimensional and complex LSFJSP, DOLDA has demonstrated excellent solving ability compared to many classic swarm intelligence algorithms. It is one of the most meaningful contributes of this work.

Table 6

The Mean and Std value of the final result of the hybrid function.

Algorithm	F21		F22		F23		F24	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	1.72E+09	7.35E+08	4.87E+02	3.40E+01	4.54E+04	7.39E+03	2.10E+04	4.05E+03
MFO	7.64E+02	3.07E+02	4.21E+02	2.12E+01	1.53E+02	3.28E+01	1.01E+02	1.67E+01
Jaya	1.25E+07	1.20E+07	2.96E+02	2.56E+01	1.36E+03	6.09E+02	6.94E+02	2.10E+02
SCA	1.44E+06	2.58E+06	2.43E+02	3.05E+01	2.87E+02	1.28E+02	1.82E+02	9.13E+01
SSA	6.92E+02	3.53E+02	1.98E+02	1.83E+01	2.24E+02	3.92E+01	1.57E+02	2.60E+01
GOA	2.59E+02	5.99E-14	2.34E+02	3.82E+01	3.36E+02	0.00E+00	1.60E+02	3.00E-14
DOLDA	1.57E+02*	2.17E-03	5.35E+01*	6.68E-04	5.03E+01*	4.37E-08	5.04E+01*	2.01E-04

Algorithm	F25		F26		F27		F28	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DA	9.85E+08	4.86E+08	1.25E+04	2.72E+03	1.31E+05	1.45E+04	5.70E+15	2.03E+15
MFO	2.41E+02	1.15E+02	1.73E+02	6.13E+00	4.36E+02	2.66E+01	1.48E+03	3.55E+03
Jaya	3.06E+06	1.31E+06	5.88E+02	1.48E+02	4.00E+03	1.03E+03	1.97E+10	3.65E+10
SCA	2.73E+05	2.65E+05	2.06E+02	2.45E+01	7.27E+02	2.47E+02	1.08E+12	1.98E+12
SSA	1.78E+02	6.35E+01	1.98E+02	1.62E+01	6.80E+02	1.16E+02	1.68E+02	4.24E+00
GOA	2.43E+04	3.83E-12	2.09E+02	3.00E-14	7.53E+02	0.00E+00	1.73E+02	0.00E+00
DOLDA	5.36E+01*	1.81E-02	1.52E+02*	1.51E-04	1.68E+02*	6.57E-04	1.59E+02*	1.65E-01

Table 7

The P values of DOLDA and other algorithms.

DOLDA										
	F1	F1	F3	F4	F5	F6	F7	F8	F9	F10
DA	-	-	-	-	-	-	-	-	-	-
MFO	+	-	-	-	+	+	-	-	+	+
Jaya	-	-	-	-	-	-	-	-	-	+
SCA	-	-	+	-	-	+	+	-	-	+
SSA	+	-	-	-	+	+	-	-	+	+
GOA	+	-	-	-	-	-	-	-	-	+

DOLDA										
	F11	F12	F13	F14	F15	F16	F17	F18	F19	F20
DA	-	-	-	-	-	+	-	-	-	-
MFO	-	-	-	-	-	+	-	-	+	-
Jaya	-	-	-	-	-	-	-	-	-	-
SCA	-	-	-	-	-	-	-	-	-	-
SSA	-	-	-	-	-	-	-	-	+	-
GOA	-	-	-	-	-	+	-	-	-	-

DOLDA										
	F21	F22	F23	F24	F25	F26	F27	F28	Same	Better
DA	-	-	-	-	-	-	-	-	1	27
MFO	-	-	-	-	-	-	-	-	7	21
Jaya	-	-	-	-	-	-	-	-	1	27
SCA	+	-	-	-	+	-	-	-	6	22
SSA	-	-	-	-	-	-	-	-	6	22
GOA	-	-	-	-	-	-	-	-	3	25

6. Conclusion

This paper applies a helpful learning strategy named DOL to enhance the performance of DA and help it to avoid suffering from premature convergence when facing the LSFJSP. The DOL strategy will enable the solution space to change dynamically and asymmetrically, which can well balance the exploitation and exploration capabilities of the algorithm to help DOLDA achieve a better solution. A total of 28 test functions from CEC 2013, 2014 as well as 15 LSFJSP instances were employed to examine the performance of DOLDA. The results obtained from the DOLDA method were compared with 6 classic swarm intelligence algorithms. Meanwhile, Two Independent-Samples T-test was conducted to evaluate the DOLDA further.

Table 8

The detail of the LSFJSP.

Instance	njob	nmac	nop	meq	aproc	mot
Instance.1	50	90	10-20	1-16	40-60	-10-10
Instance.2				17-32		
Instance.3				33-48		
Instance.4	100	90	10-20	1-16	40-60	-10-10
Instance.5				17-32		
Instance.6				33-48		
Instance.7	100	120	10-20	1-24	40-60	-10-10
Instance.8				25-48		
Instance.9				49-72		
Instance.10	150	120	10-20	1-24	40-60	-10-10
Instance.11				25-48		
Instance.12				49-72		
Instance.13	200	120	10-20	1-24	40-60	-10-10
Instance.14				25-48		
Instance.15				49-72		

njob: number of jobs;

nmac: number of machines;

nop: minimum and maximum number of operation per job;

meq: minimum and maximum number of available machine per operation;

aproc: average processing time per operation;

mot: offset time boundaries on per machine.

The results of all test functions demonstrate the competitive superiority of proposed DOLDA in solving the global optimization of these unimodal and multimodal problems. In addition, further trials on 15 LSFJSP instances report that DOLDA can handle actual large-scale problems. Besides, this paper constructs a novel time-saving encoding and decoding principle to increase the speed of solving LSFJSP. The jumping rate is an essential factor that influence the DOL strategy. For this end, the reasonable jumping rate analyzed in this paper has provided a valuable reference for the users in future researches and applications.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research work is supported by the National Key Research and Development Project under Grant 2018YFB1700500.

Table 9

The experiment results of SM and AM.

Instance	DOLDA		GOA		SSA		SCA		Jaya		MFO		DA	
	SM	AM	SM	SM	SM	AM	AM	AM	SM	SM	SM	AM	AM	AM
Instance.1	1236	1253	1279	1294	1249	1261	1255	1266	1251	1264	1273	1275	1271	1290
Instance.2	1232	1251	1241	1255	1236	1253	1244	1253	1255	1259	1239	1259	1249	1259
Instance.3	1175	1223	1228	1261	1195	1240	1175	1235	1169	1236	1181	1242	1230	1261
Instance.4	1670	1679	1680	1694	1650	1691	1673	1683	1680	1690	1694	1712	1671	1691
Instance.5	1654	1674	1651	1684	1677	1693	1677	1686	1682	1692	1708	1718	1682	1697
Instance.6	1657	1685	1701	1705	1658	1690	1672	1696	1676	1686	1701	1705	1669	1685
Instance.7	1461	1485	1470	1528	1462	1497	1468	1494	1477	1495	1465	1495	1463	1532
Instance.8	1428	1495	1457	1499	1438	1493	1442	1498	1452	1498	1467	1523	1461	1495
Instance.9	1459	1479	1443	1481	1480	1495	1478	1492	1472	1495	1484	1503	1478	1493
Instance.10	1768	1807	1814	1824	1790	1814	1788	1816	1817	1824	1811	1835	1797	1814
Instance.11	1732	1807	1806	1820	1801	1819	1794	1815	1793	1814	1796	1829	1756	1808
Instance.12	1793	1835	1844	1856	1826	1849	1814	1837	1817	1834	1822	1860	1805	1833
Instance.13	2091	2124	2121	2143	2140	2153	2143	2156	2132	2153	2125	2159	2133	2151
Instance.14	2139	2153	2098	2170	2143	2181	2142	2172	2146	2183	2153	2187	2151	2172
Instance.15	2133	2153	2134	2156	2126	2160	2149	2159	2152	2165	2151	2179	2122	2168
Best num	9	13	3	0	1	1	0	0	1	0	0	0	1	2

Best num represents the number of the best solution.

Table 10

The experiment results of SD and AD.

Instance	DOLDA		GOA		SSA		SCA		Jaya		MFO		DA	
	SD	AD	SD	AD	SD	AD	SD	AD	SD	AD	SD	AD	SD	AD
Instance.1	−13%	−11%	11%	12%	−6%	−6%	−2%	−3%	−5%	−4%	7%	2%	7%	10%
Instance.2	−6%	−3%	−1%	0%	−4%	−1%	1%	−1%	7%	2%	−2%	2%	4%	2%
Instance.3	−11%	−11%	20%	10%	1%	−1%	−11%	−4%	−15%	−4%	−7%	0%	21%	10%
Instance.4	−2%	−5%	3%	1%	−10%	0%	0%	−4%	3%	−1%	8%	8%	−1%	0%
Instance.5	−9%	−8%	−11%	−3%	0%	0%	0%	−2%	3%	0%	13%	11%	3%	2%
Instance.6	−8%	−3%	10%	5%	−8%	−1%	−2%	1%	0%	−3%	10%	5%	−3%	−3%
Instance.7	−3%	−9%	2%	11%	−2%	−3%	1%	−5%	5%	−4%	−1%	−4%	−2%	13%
Instance.8	−10%	−2%	4%	−1%	−5%	−3%	−4%	−1%	1%	−1%	8%	11%	6%	−2%
Instance.9	−6%	−6%	−13%	−5%	4%	2%	4%	0%	1%	2%	6%	6%	4%	1%
Instance.10	−12%	−5%	6%	2%	−3%	−2%	−4%	−1%	7%	2%	5%	6%	0%	−2%
Instance.11	−20%	−3%	9%	2%	7%	1%	4%	0%	4%	−1%	5%	5%	−11%	−3%
Instance.12	−9%	−3%	10%	5%	3%	2%	−1%	−2%	0%	−4%	2%	6%	−5%	−4%
Instance.13	−12%	−8%	−2%	−2%	4%	1%	5%	2%	2%	1%	0%	3%	2%	1%
Instance.14	0%	−7%	−14%	−1%	1%	2%	1%	−1%	2%	3%	5%	4%	4%	−1%
Instance.15	−2%	−3%	−1%	−2%	−4%	−1%	4%	−1%	5%	1%	4%	5%	−5%	2%
Best num	9	13	3	0	2	1	0	0	1	2	0	0	1	3

Best num represents the number of the best solution.

References

- [1] A. Letafat, M. Rafiei, M. Ardeshtari, M. Sheikh, M. Banaei, J. Boudjadar, M.H. Khooban, An efficient and cost-effective power scheduling in zero-emission ferry ships, *Complexity* 2020 (2020) <http://dx.doi.org/10.1155/2020/6487873>.
- [2] L. Davis, Job shop scheduling with genetic algorithms, in: *Proceedings of an International Conference on Genetic Algorithms and their Applications*, Vol. 140, 1985.
- [3] P. Brucker, B. Jurisch, B. Sievers, A branch and bound algorithm for the job-shop scheduling problem, *Discrete Appl. Math.* 49 (1–3) (1994) 107–127.
- [4] J.F. Muth, G.L. Thompson, *Industrial Scheduling*, Prentice-Hall, 1963.
- [5] J. Carlier, E. Pinson, An algorithm for solving the job-shop problem, *Manage. Sci.* 35 (2) (1989) 164–176.
- [6] H. Mokhtari, A. Hasani, An energy-efficient multi-objective optimization for flexible job-shop scheduling problem, *Comput. Chem. Eng.* 104 (2017) 339–352.
- [7] Y. Wang, et al., A new hybrid genetic algorithm for job shop scheduling problem, *Comput. Oper. Res.* 39 (10) (2012) 2291–2299.
- [8] D. Gao, G.-G. Wang, W. Pedrycz, Solving fuzzy job-shop scheduling problem using DE algorithm improved by a selection mechanism, *IEEE Trans. Fuzzy Syst.* 28 (12) (2020) 3265–3275.
- [9] M. Yu, T. Quan, Q. Peng, X. Yu, L. Liu, A model-based collaborative filtering algorithm based on stacked AutoEncoder, *Neural Comput. Appl.* (2021) 1–9.
- [10] K.Z. Gao, P.N. Suganthan, Q.K. Pan, M.F. Tasgetiren, A. Sadollah, Artificial bee colony algorithm for scheduling and rescheduling fuzzy flexible job shop problem with new job insertion, *Knowl.-Based Syst.* 109 (2016) 1–16.
- [11] H. Ding, X. Gu, Improved particle swarm optimization algorithm based novel encoding and decoding schemes for flexible job shop scheduling problem, *Comput. Oper. Res.* 121 (2020) 104951.
- [12] T. Ning, H. Jin, X. Song, B. Li, An improved quantum genetic algorithm based on MAGTD for dynamic FJSP, *J. Ambient Intell. Humaniz. Comput.* 9 (4) (2018) 931–940.
- [13] M.M. Ahmadian, A. Salehipour, T. Cheng, A meta-heuristic to solve the just-in-time job-shop scheduling problem, *European J. Oper. Res.* 288 (1) (2021) 14–29.
- [14] X. Gu, M. Huang, X. Liang, An improved genetic algorithm with adaptive variable neighborhood search for FJSP, *Algorithms* 12 (11) (2019) 243.
- [15] M. Mahmoodjanloo, R. Tavakkoli-Moghaddam, A. Baboli, A. Bozorgi-Amiri, Flexible job shop scheduling problem with reconfigurable machine tools: An improved differential evolution algorithm, *Appl. Soft Comput.* 94 (2020) 106416.
- [16] A. Vital-Soto, A. Azab, M.F. Baki, Mathematical modeling and a hybridized bacterial foraging optimization algorithm for the flexible job-shop scheduling problem with sequencing flexibility, *J. Manuf. Syst.* 54 (2020) 74–93.
- [17] X. Yu, F. Jiang, J. Du, D. Gong, A cross-domain collaborative filtering algorithm with expanding user and item features via the latent factor space of auxiliary domains, *Pattern Recognit.* 94 (2019) 96–109.
- [18] G. Dhiman, M. Garg, A. Nagar, V. Kumar, M. Dehghani, A novel algorithm for global optimization: Rat swarm optimizer, *J. Ambient Intell. Humaniz. Comput.* (2020) 1–26.
- [19] J. Lin, L. Zhu, Z.-J. Wang, A hybrid multi-verse optimization for the fuzzy flexible job-shop scheduling problem, *Comput. Ind. Eng.* 127 (2019) 1089–1100.
- [20] J. Sun, G. Zhang, J. Lu, W. Zhang, A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times, *Comput. Oper. Res.* 132 (2021) 105263.
- [21] X. Yu, Y. Chu, F. Jiang, Y. Guo, D. Gong, SVMs classification based two-side cross domain collaborative filtering by inferring intrinsic user and item features, *Knowl.-Based Syst.* 141 (2018) 80–91.

- [22] Z. Beheshti, S.M.H. Shamsuddin, A review of population-based meta-heuristic algorithms, *Int. J. Adv. Soft Comput. Appl.* 5 (1) (2013) 1–35.
- [23] X.-q. Shi, W. Long, Y.-y. Li, Y.-l. Wei, D.-s. Deng, Different performances of different intelligent algorithms for solving FJSP: a perspective of structure, *Comput. Intell. Neurosci.* 2018 (2018).
- [24] A.B. Hmida, M. Haouari, M.-J. Huguet, P. Lopez, Discrepancy search for the flexible job shop scheduling problem, *Comput. Oper. Res.* 37 (12) (2010) 2192–2201.
- [25] X. Wang, K. Xing, C.-B. Yan, M. Zhou, A novel MOEA/D for multiobjective scheduling of flexible manufacturing systems, *Complexity* (2019) <http://dx.doi.org/10.1155/2019/5734149>.
- [26] F. Pezzella, G. Morganti, G. Ciaschetti, A genetic algorithm for the flexible job-shop scheduling problem, *Comput. Oper. Res.* 35 (10) (2008) 3202–3212.
- [27] L. Wang, G. Zhou, Y. Xu, S. Wang, M. Liu, An effective artificial bee colony algorithm for the flexible job-shop scheduling problem, *Int. J. Adv. Manuf. Technol.* 60 (2012) 303–315.
- [28] X. Wu, X. Liu, N. Zhao, An improved differential evolution algorithm for solving a distributed assembly flexible job shop scheduling problem, *Memetic Comput.* (2019) 1–21.
- [29] J.-Q. Li, P. Duan, J. Cao, X. Lin, Y. Han, A hybrid Pareto-based Tabu search for the distributed flexible job shop scheduling problem with E/T criteria, *IEEE Access* 6 (2018) 58883–58897.
- [30] Y. Xu, L. Wang, S. Wang, M. Liu, An effective teaching-learning-based optimization algorithm for the flexible job-shop scheduling problem with fuzzy processing time, *Neurocomputing* 148 (2015) 260–268.
- [31] S. Mirjalili, Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems, *Neural Comput. Appl.* 27 (4) (2016) 1053–1073.
- [32] V. Suresh, S. Sreejith, Generation dispatch of combined solar thermal systems using dragonfly algorithm, *Computing* 99 (1) (2017) 59–80.
- [33] P.T. Daely, S.Y. Shin, Range based wireless node localization using dragonfly algorithm, in: 2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN), IEEE, 2016, pp. 1012–1015.
- [34] M. Jafari, M.H.B. Chaleshtari, Using dragonfly algorithm for optimization of orthotropic infinite plates with a quasi-triangular cut-out, *Eur. J. Mech. A Solids* 66 (2017) 1–14.
- [35] C.M. Rahman, T.A. Rashid, Dragonfly algorithm and its applications in applied science survey, *Comput. Intell. Neurosci.* 2019 (2019).
- [36] L. Zhong, Y. Zhou, Q. Luo, K. Zhong, Wind driven dragonfly algorithm for global optimization, *Concurr. Comput.: Pract. Exper.*
- [37] W. Dai, C. Li, Z. Cui, Y. Wu, J. Huang, An improved dragonfly algorithm with higher exploitation capability to optimize the design of hybrid power active filter, *IEEE Access* PP (99) (2020) 1.
- [38] S. Debnath, S. Baishya, D. Sen, W. Arif, A hybrid memory-based dragonfly algorithm with differential evolution for engineering application, *Engineering with Computers* (2020) 1–28.
- [39] L.-L. Li, X. Zhao, M.-L. Tseng, R.R. Tan, Short-term wind power forecasting based on support vector machine with improved dragonfly algorithm, *Journal of Cleaner Production* 242 (2020) 118447.
- [40] X. Cui, Y. Li, J. Fan, T. Wang, Y. Zheng, A hybrid improved dragonfly algorithm for feature selection, *IEEE Access* 8 (2020) 155619–155629.
- [41] Y. Xu, Z. Yang, X. Li, H. Kang, X. Yang, Dynamic opposite learning enhanced teaching-learning-based optimization, *Knowledge-Based Systems* 188 (2020) 104966.
- [42] R. Ruiz, J.A. Vázquez-Rodríguez, The hybrid flow shop scheduling problem, *European J. Oper. Res.* 205 (1) (2010) 1–18.
- [43] M. Wu, D. Yang, B. Zhou, Z. Yang, T. Liu, L. Li, Z. Wang, K. Hu, Adaptive population nsga-iii with dual control strategy for flexible job shop scheduling problem with the consideration of energy consumption and weight, *Machines* 188 (2020) 104966.
- [44] M. Shahgholi Zadeh, Y. Katebi, A. Doniavi, A heuristic model for dynamic flexible job shop scheduling problem considering variable processing times, *Int. J. Prod. Res.* 57 (10) (2019) 3020–3035.
- [45] R.H. Caldeira, A. Gnanavelbabu, T. Vaidyanathan, An effective backtracking search algorithm for multi-objective flexible job shop scheduling considering new job arrivals and energy consumption, *Comput. Ind. Eng.* 149 (2020) 106863.
- [46] H.R. Tizhoosh, Opposition-based learning: a new scheme for machine intelligence, in: *International Conference on Computational Intelligence for Modelling, Control and Automation and International Conference on Intelligent Agents, Web Technologies and Internet Commerce (CIMCA-IAWTIC'06)*, Vol. 1, IEEE, 2005, pp. 695–701.
- [47] G. Zhang, L. Gao, Y. Shi, An effective genetic algorithm for the flexible job-shop scheduling problem, *Expert Systems with Applications* 38 (4) (2011) 3563–3573.
- [48] K. Mesghouni, S. Hammadi, P. Borne, Evolution programs for job-shop scheduling, in: 1997 IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation, Vol. 1, IEEE, 1997, pp. 720–725.
- [49] P. Fattahi, M.S. Mehrabad, F. Jolai, Mathematical modeling and heuristic approaches to flexible job shop scheduling problems, *Journal of intelligent manufacturing* 18 (3) (2007) 331–342.
- [50] J. Liang, B. Qu, P. Suganthan, A.G. Hernández-Díaz, Problem definitions and evaluation criteria for the CEC 2013 special session on real-parameter optimization, in: *Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou, China and Nanyang Technological University, Singapore, Technical Report*, 201212, (34) 2013, pp. 281–295.
- [51] J.J. Liang, B.Y. Qu, P.N. Suganthan, Problem definitions and evaluation criteria for the CEC 2014 special session and competition on single objective real-parameter numerical optimization, in: *Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou China and Technical Report*, Nanyang Technological University, Singapore, 635, 2013, p. 490.
- [52] S. Mirjalili, Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems, *Neural Comput. Appl.* 27 (4) (2016) 1053–1073.
- [53] S. Mirjalili, Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm, *Knowl.-Based Syst.* 89 (2015) 228–249.
- [54] R. Rao, Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems, *Int. J. Ind. Eng. Comput.* 7 (1) (2016) 19–34.
- [55] S. Mirjalili, SCA: A Sine cosine algorithm for solving optimization problems, *Knowl.-Based Syst.* 96 (2016).
- [56] S. Mirjalili, A.H. Gandomi, S.Z. Mirjalili, S. Saremi, H. Faris, S.M. Mirjalili, Salp swarm algorithm: A bio-inspired optimizer for engineering design problems, *Adv. Eng. Softw.* 114 (2017) 163–191.
- [57] S. Saremi, S. Mirjalili, A. Lewis, Grasshopper optimisation algorithm: theory and application, *Adv. Eng. Softw.* 105 (2017) 30–47.
- [58] P. Brandimarte, Routing and scheduling in a flexible job shop by tabu search, *Ann. Oper. Res.* 41 (3) (1993) 157–183.